

Image generated with Microsoft Copilot AI Tool.

Predicing Asset Price Direction

Dr Richard Diamond, PhD, CQF, ARPM

Point 1. **Limited Forecasting and Risk** This demonstration is a stylised example of running classifiers (supervised learning) on equity returns. We limit our task to prediction of daily return direction (sign), which is necessary in order to achieve a limited degree of success.

75-80% traders in Contract for Difference (CDF) routinely lose their deposits 1. CFDs is a retail terminology for Total Return Swaps, lucrative business for large banks except for Archegos debacle 2021. Retail clients also have very limited success with binary options, which are long de-listed from professional exchanges. If there were a reliable way to forecast daily index returns, or at least the direction..

Point 2. **Asset Pricing Theory** There is a misconception about the naming and purpose. APT is not concerned with price prediction but the 'pricing' in relation to the market benchmark (S&P 500 index). APT concerns itself estimation of beta(s) estimating of beta, which is relation between asset returns and the market. First, if we know the beta, we know where we are on in relation to markets. Second, beta estimation can be made more robust than daily returns.

One is more likely to find a sensible scheme in predicting specialised assets and co-movement (spread) rather than returns (prices) of an individual assets.

Here, we consider a large-cap equities, however in practice, the high frequency returns of large caps and market indices are the most difficult quantities to predict.

Point 3. CQF program likely to have already covered the more nuansed use of KNN SVR for Stock Prediction (see Python Labs). There are also Introduction to ML using Scikit-learn and Algorithmic Trading Strategy type of labs. Consider this tutorial Statistical Essentials side that explains **Likelihood Approach** — we remind of linear regression and consider the likelihood in logistic classifier setting, where it's computed using the logistic sigmoid function.

Improving on price information

- To organise the prediction we start with features: generate 7-10 price-related features (columns) and run Classifier.fit() on them. For each classifier, you can produce ROC, Confusion Matrix, Transition Probabilities -- the startard metrics of supervised learning.

- This demo derives features (lagged returns) from the asset price. There is large discussion about how much information is present in stock prices vs. other data, such as fundamentals, macro economic, and alternative data.

However, the forecasting can be technically improved by using the measures of Average and Momentum. You will need to make reasonable choices of **time period** (rolling window) for SMA, EMA, Momentum. This link, gives one example on how to do Exponential Moving Average from the first principles. The choice of smoothing factor $\alpha = 2/(N_{obs} + 1)$.

Binomial or Multinomial?

Setting up prediction classes, you are likely to encounter the high count of uncertain moves/'no move', where the movement took less than 1/10th of a percentage point.

Remember that for a global market index, the daily average return can be $O(0.04\%)$. $[-1, 0, 1]$ trinomial classification might be more appropriate, but there is **no trading action** that follows from the prediction of no move in the asset.

One but not a universal approach, is to label 'no moves' as positive moves. That keeps continuity because most of the moves are positive and we are interested in:

```
1) isolating information carried by empirical negative
returns;

2) better quality in negative market moves prediction.
```

Because of the asymmetry of reward for losing money (which one wants to avoid and associated with worse feelings), as compared to gaining money.

Simply dropping 'no move' observations will be detrimental in such sensitive tasks as daily predition.

NOT Least Squares

- This demo is **NOT** about autoregression analysis in itself. Autoregression is the application of regression on past returns. The regression is essentially an application a linear model and the method of computation (Maximum Likelihood) involves "the Ordinary Least Squares", $\epsilon_t^2 = (y_t - \hat{\boldsymbol{\beta}}\boldsymbol{x_t})^2$, hence the abbreviation OLS.

If you formally attempt to use the autogression (or vector autoregression for multiple tickers at the same time) in order to predict the daily return *quantity* -- you will encounter the forecasting error of $150 - 200\%$.

```
Example: you are predicting return of -1.5%, the order of
error above means that realised return will be anything up
to +1.5%, eg 3% absolute difference.
```

- OLS is **NOT** a valid regression model for binary dependent variable $0, 1$ or for y_t, x_t any other than Normal variables. There is no inherent model risk when OLS is applied to **quarterly changes** in such variables as GDP and economic macro -- that is a valid forecasting move. However, OLS does fail at predicing financial asset daily returns (eg, error of $150 - 200\%$) and is certainly methodologically wrong to use on non-stationary, stochastic prices S_t. Think about it: when you are correlating and regressing prices of two assets, you simply regressing one Brownian Motion on another.

Change in the nature of dependent variable leads to the change in Maximum Likelhood -- which is 'under the hood' method of how regression works, how regression coefficients are derived and computed. We will consider that today.

Genearalized Linear Models (GLM) is both, extension and the substitution of the linear regression (OLS). It implies a non-linear **link function** between dependent variable, such as Binary $0, 1$ and implied probability $[0.01..0.99]$. Logistic Regression (Classification) and its Maximum Likelihood is a good illustration and introduction to the GLM.
We will also demonstrate that prediction results (for asset price direction) that are

coming from OLS linear regression are *a fudge*.

In [21]:
```
import numpy as np
import pandas as pd

from pylab import plt
plt.style.use("seaborn-v0_8")
%matplotlib inline

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

Generating simple features: lagged returns

In [2]:
```
data = pd.read_excel('data/EquitiesDataGOOG.xlsx', sheet_name="Sheet1", i
data.columns = ['price']  # Adj Close Price
```

In [48]:
```
data.head()
```

Out[48]:

	price
Date	
2010-09-30	261.91
2010-10-01	261.83
2010-10-04	260.20
2010-10-05	268.11
2010-10-06	266.18

In [49]:
```
data.plot(figsize=(10, 6));
```



In [3]:
```
# SEPARATE for Features Correlation
data_advf = data.copy() #save a copy for advanced look at lagged returns

data_advf['Returns'] = np.log(data_advf).diff()

data_advf.head()
```

Out[3]:

Date	price	Returns
2010-09-30	261.91	NaN
2010-10-01	261.83	-0.000305
2010-10-04	260.20	-0.006245
2010-10-05	268.11	0.029947
2010-10-06	266.18	-0.007225

In [4]:
```
data['return'] = np.log(data / data.shift(1))
```

In [52]:
```
data.head()
```

Out[52]:

Date	price	return
2010-09-30	261.91	NaN
2010-10-01	261.83	-0.000305
2010-10-04	260.20	-0.006245
2010-10-05	268.11	0.029947
2010-10-06	266.18	-0.007225

In [5]:
```
lags = 5

cols = []
for lag in range(1, lags+1):
    col = 'ret_%d' % lag
    data[col] = data['return'].shift(lag)
    cols.append(col)

# Column indexation is from 0, so we effectively have range(1, 6) for ret
# CHECK NOT TO USE Return IN ACTUAL PREDICTION
```

In [54]:
```
data.head(10)
```

Out[54]:

Date	price	return	ret_1	ret_2	ret_3	ret_4	ret_5
2010-09-30	261.91	NaN	NaN	NaN	NaN	NaN	NaN
2010-10-01	261.83	-0.000305	NaN	NaN	NaN	NaN	NaN
2010-10-04	260.20	-0.006245	-0.000305	NaN	NaN	NaN	NaN
2010-10-05	268.11	0.029947	-0.006245	-0.000305	NaN	NaN	NaN
2010-10-06	266.18	-0.007225	0.029947	-0.006245	-0.000305	NaN	NaN
2010-10-07	264.02	-0.008148	-0.007225	0.029947	-0.006245	-0.000305	NaN
2010-10-08	267.17	0.011860	-0.008148	-0.007225	0.029947	-0.006245	-0.000305
2010-10-11	268.41	0.004631	0.011860	-0.008148	-0.007225	0.029947	-0.006245
2010-10-12	269.68	0.004720	0.004631	0.011860	-0.008148	-0.007225	0.029947
2010-10-13	270.64	0.003553	0.004720	0.004631	0.011860	-0.008148	-0.007225

In [6]:
```
data.dropna(inplace=True)

data = data.drop(columns="price")

data['return_sign'] = np.sign(data['return'].values)

data_adv = data.copy() #save a copy for advanced methods (classifiers), a
```

In [57]:
```
data
```

Out[57]:

Date	return	ret_1	ret_2	ret_3	ret_4	ret_5	return_s
2010-10-08	0.011860	-0.008148	-0.007225	0.029947	-0.006245	-0.000305	
2010-10-11	0.004631	0.011860	-0.008148	-0.007225	0.029947	-0.006245	
2010-10-12	0.004720	0.004631	0.011860	-0.008148	-0.007225	0.029947	
2010-10-13	0.003553	0.004720	0.004631	0.011860	-0.008148	-0.007225	
2010-10-14	-0.004370	0.003553	0.004720	0.004631	0.011860	-0.008148	
...	...	...	...	...	...	...	
2017-09-25	-0.008175	-0.004213	0.000933	0.010543	0.007415	-0.005765	
2017-09-26	0.004215	-0.008175	-0.004213	0.000933	0.010543	0.007415	
2017-09-27	0.021003	0.004215	-0.008175	-0.004213	0.000933	0.010543	
2017-09-28	0.005290	0.021003	0.004215	-0.008175	-0.004213	0.000933	
2017-09-29	0.010070	0.005290	0.021003	0.004215	-0.008175	-0.004213	

1757 rows × 7 columns

Autocorrelation in Lagged Returns (long run)

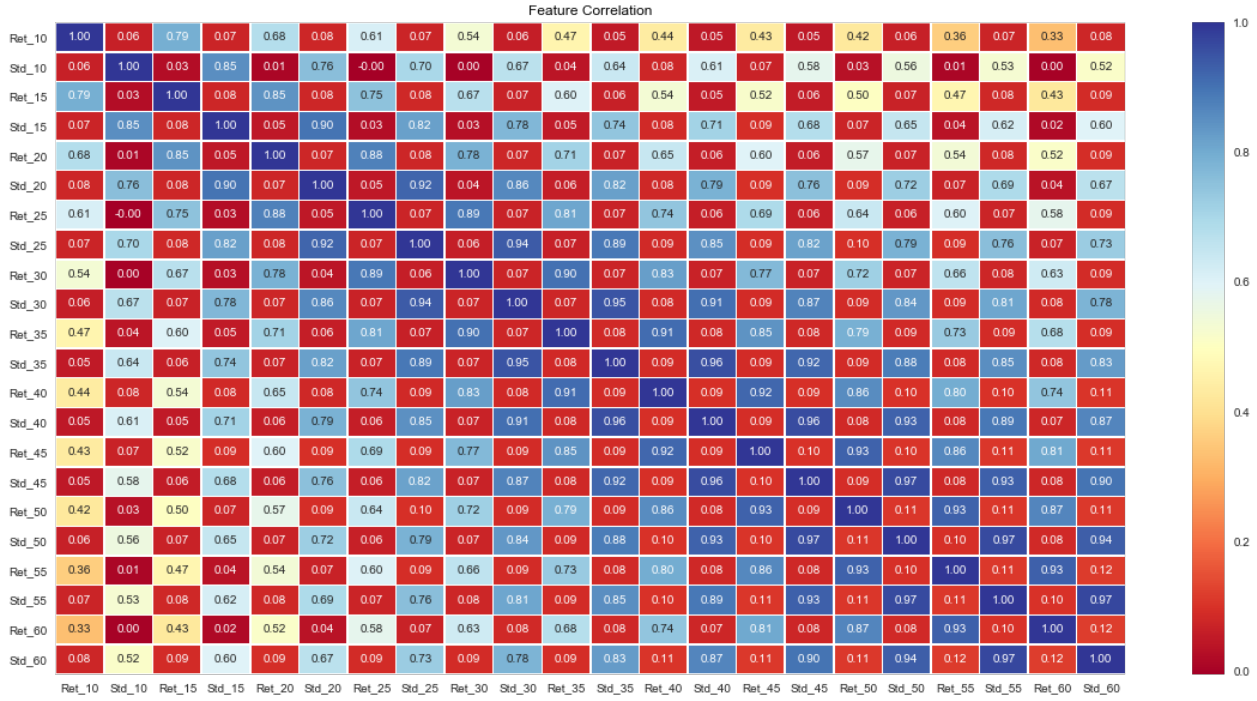
In [27]:
```
# Create features (predictors) list
features_list = []
for r in range(10, 65, 5):
    data_advf['Ret_'+str(r)] = data_advf.Returns.rolling(r).sum()
    data_advf['Std_'+str(r)] = data_advf.Returns.rolling(r).std()
    features_list.append('Ret_'+str(r))
    features_list.append('Std_'+str(r))

# Drop NaN values
```

```
data_advf.dropna(inplace=True)
```

In [28]:
```
# Derive features correlation
import seaborn as sns
corrmat = data_advf.drop(['price', 'Returns'],axis=1).corr()

# Visualize feature correlation
fig, ax = plt.subplots(figsize=(20,10))
sns.heatmap(corrmat, annot=True, annot_kws={"size": 10}, fmt="0.2f", line
ax.set_title('Feature Correlation', fontsize=12, color='black');
```



Maximum Likelihood for Regression (Ordinary Least Squares)

Below work is illustration-only. Do not run OLS in your assignments and projects.

When the assumption of Normality of residuals holds: ϵ_t is *iid* $N(0, \sigma^2)$, **the linear regression** $y_t = \hat{\boldsymbol{\beta}} \boldsymbol{x_t} + \epsilon_t$ has MLE properties.

That means estimated coefficients $\hat{\boldsymbol{\beta}}$ are

- consistent (i.e., close to unknown true estimates β with low tolerance) and
- asymptotically efficient (i.e., their variance is known and minimised).

Estimates $\hat{\boldsymbol{\beta}}$ in fact, maximise the following **joint Normal** likelihood:

$$\mathbf{L} = \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right)^T \exp -\frac{1}{2} \left[\frac{\epsilon_1^2}{\sigma^2} + \frac{\epsilon_2^2}{\sigma^2} + \cdots + \frac{\epsilon_T^2}{\sigma^2} \right]$$

Substituting $\epsilon_t = y_t - \hat{\boldsymbol{\beta}}\boldsymbol{x_t}$ and taking log gives for an individual observation -- I call this quantity a contribution of likelihood from an observation (data row values of features) at time *t*

$$\log L_t = -\frac{1}{2}\log 2\pi\sigma^2 - \frac{1}{2}\frac{(y_t - \hat{\boldsymbol{\beta}}\boldsymbol{x_t})^2}{\sigma^2}$$

Total log-likelihood for a regression model is the sum of contributions from each observation $\log \mathbf{L} = \sum_{t=1}^{T} L_t$.

Numerical MLE varies $\hat{\boldsymbol{\beta}}$ to maximise $\log \mathbf{L}$. This can be done by any non-specific optimisation routine, such as Excel Solver. It is clear to spot that log-likelihood is maximised by **minimising** the residual sum of squares

$$RSS = \sum_{t=1}^{T} \epsilon_t^2 = \sum_{t=1}^{T} (y_t - \hat{\boldsymbol{\beta}}\boldsymbol{x_t})^2$$

.

Residual sum of squares (RSS) is also known as the sum of squared residuals (SSR) or the sum of squared estimate of errors (SSE).

CAUTION OLS is an **invalid** model for binary dependent variable {0, 1}. Think about a change in MLE function for such variable.

In [7]:
```
# Regression from NUMPY library
reg_coef = np.linalg.lstsq(data[cols].values, data['return_sign'])[0]

# PREFER delegates to use STATSMOTELS
#import statsmodels.api as sm
```

```
/var/folders/w5/cglfkxqx0td548s77k854nmc0000gn/T/ipykernel_32336/388275098
1.py:2: FutureWarning: `rcond` parameter will change to the default of mac
hine precision times ``max(M, N)`` where M and N are the input matrix dime
nsions.
To use the future default and silence this warning we advise to pass `rcon
d=None`, to keep using the old, explicitly pass `rcond=-1`.
  reg_coef = np.linalg.lstsq(data[cols].values, data['return_sign'])[0]
```

In [14]:
```
reg_coef
```

Out[14]:
```
array([ 1.13255353,  0.34409898, -2.87238464, -0.40975749, -1.38671844])
```

In [8]:
```
data['ols_pred'] = np.sign(np.dot(data[cols].values, reg_coef)) #dot prod
```

In [16]:
```
data.head(15)
```

Out[16]:

Date	return	ret_1	ret_2	ret_3	ret_4	ret_5	return_
2010-10-08	0.011860	-0.008148	-0.007225	0.029947	-0.006245	-0.000305	
2010-10-11	0.004631	0.011860	-0.008148	-0.007225	0.029947	-0.006245	
2010-10-12	0.004720	0.004631	0.011860	-0.008148	-0.007225	0.029947	
2010-10-13	0.003553	0.004720	0.004631	0.011860	-0.008148	-0.007225	
2010-10-14	-0.004370	0.003553	0.004720	0.004631	0.011860	-0.008148	
2010-10-15	0.106028	-0.004370	0.003553	0.004720	0.004631	0.011860	
2010-10-18	0.026677	0.106028	-0.004370	0.003553	0.004720	0.004631	
2010-10-19	-0.016119	0.026677	0.106028	-0.004370	0.003553	0.004720	
2010-10-20	0.000231	-0.016119	0.026677	0.106028	-0.004370	0.003553	
2010-10-21	0.006582	0.000231	-0.016119	0.026677	0.106028	-0.004370	
2010-10-22	0.000885	0.006582	0.000231	-0.016119	0.026677	0.106028	
2010-10-25	0.006468	0.000885	0.006582	0.000231	-0.016119	0.026677	
2010-10-26	0.003381	0.006468	0.000885	0.006582	0.000231	-0.016119	
2010-10-27	-0.003446	0.003381	0.006468	0.000885	0.006582	0.000231	
2010-10-28	0.003446	-0.003446	0.003381	0.006468	0.000885	0.006582	

Counts of predicted moves DOWN and UP

In [17]:
```
data['ols_pred'].value_counts()

#c.value_counts()[1] / (c.value_counts().sum()) #We were UP this percenta
```

Out[17]:
```
-1.0    938
 1.0    819
Name: ols_pred, dtype: int64
```

False Negatives -- Type II Error

The probelm transpires from counts above: OLS predicts a lot of *false negatives* with -1 label. We know that the proportion of noticeably negative returns is smaller than $\frac{938}{938+819} \approx 53.4\%$. E.g., returns not the order of -0.04% but of the larger negative magnitude such -0.5%.

In terms of the generated asset path (cumulative P&L of hold position): in case of the bad prediction of negative returns (moves down) we would observe the path drifting downwards and downwards.

Vectorised Backtesting = Rebalancing

'ols_pred'	'return'	Result P&L
NEGATIVE 'ols_pred'	NEGATIVE 'return'	POSITIVE P&L
NEGATIVE 'ols_pred'	POSITIVE 'return' (move up)	NEGATIVE P&L, Loss
POSITIVE 'ols_pred'	NEGATIVE 'return'	NEGATIVE P&L, Loss
POSITIVE 'ols_pred'	POSITIVE 'return' (move up)	POSITIVE P&L

Exercise care with Dr Hilpisch code, particularly on 'vectorised backtesting': where correctly predicted negative sign translates into the Positive P&L.

The assumption is daily **rebalancing** (daily betting) +100% or -100% of the asset, rather than replication of the actual asset path (hold position).

In [9]:
```
data['ols_pred_backtest'] = data['return'] * data['ols_pred']
```

So, the vectorised backtesting assumes/represents a sequence of daily bets, based on the sign (up/down move) predicted from past returns.

The actual return that realises today t, is % P&L that one makes (loses) on the bet of $100, for example if return today is POSITIVE 0.018 and predcited sign was POSITIVE, then P&L is 1.18%.

$$Return_{pred} = Z^* \times \sigma$$

Assume we want to have more model-like prediction. Then, we will use standard deviation, which gives some measure of randomness.

- We use std dev from the dataset but that can be estimated from any prevoius holdout period/window. Such backtesting requires past data but not regular daily update of return -- the latter is historical backtesting.
- $Z^* = \pm 0.7$ of the standard deviation translates into betting %70 of one sigma -- the Actual P&L which would still depend on actual return. But we can plot cumulative P&L from 'ols_pred_move' to see if matches with the asset path. It also is possible to do P&L Attribution test on such std dev model:

'ols_pred_move' is in effect, our **Theoretical P&L**

'return' is the **Actual P&L**

- Why z-score of ± 0.7? This is because empirical asset returns are not well-Normal and between ± 2 standard deviation, but their density/histogram is high-peak (high mode). If you standardise returns $z_t = (r_t - \mu)/\sigma$ the histogram of z_t will have bars within ± 0.7.
- Instead of ± 0.7 and to provide negative outcomes, we can use simulated values of Random Normal ϕ which can be positive or negative.

In [10]:
```
stdev = data['return'].std()  #from a whole dataset but you can introduce

data['ols_pred_move'] = 0.7 * stdev * (data['ols_pred'])
data['ols_pred_move_FUDGED'] = 0.7 * stdev * ( - data['ols_pred']) #PREDI

data['ols_pred_ABSOLUTE'] = abs(data['return']) * data['ols_pred']


# Econometricans would simulate the Standard Normal, instead of 0.7, howe
# Emplirical returns are tightly clustered in the small move/no move regi
```

In [151…
```
data.head(10)
```

Out[151...

Date	return	ret_1	ret_2	ret_3	ret_4	ret_5	return_s
2010-10-08	0.011860	-0.008148	-0.007225	0.029947	-0.006245	-0.000305	
2010-10-11	0.004631	0.011860	-0.008148	-0.007225	0.029947	-0.006245	
2010-10-12	0.004720	0.004631	0.011860	-0.008148	-0.007225	0.029947	
2010-10-13	0.003553	0.004720	0.004631	0.011860	-0.008148	-0.007225	
2010-10-14	-0.004370	0.003553	0.004720	0.004631	0.011860	-0.008148	
2010-10-15	0.106028	-0.004370	0.003553	0.004720	0.004631	0.011860	
2010-10-18	0.026677	0.106028	-0.004370	0.003553	0.004720	0.004631	
2010-10-19	-0.016119	0.026677	0.106028	-0.004370	0.003553	0.004720	
2010-10-20	0.000231	-0.016119	0.026677	0.106028	-0.004370	0.003553	
2010-10-21	0.006582	0.000231	-0.016119	0.026677	0.106028	-0.004370	

In [16]:
```
data[['return', 'ols_pred_ABSOLUTE', 'ols_pred_move']].cumsum().apply(np.
```



- 'ols_pred_ABSOLUTE' and 'ols_pred_move' from std dev do not reproduce the asset price path at all. This plot reveals the poor quality of prediction. **FINDING:** correct negatives plus false negatives make for a bad prediction. OLS seems to produce a lot of negative predicitons which do not realise.

In [60]:
```
data[['return', 'ols_pred_backtest', 'ols_pred_move_FUDGED']].cumsum().ap
```



INTERIM QUESTIONS

- Why 'pred_return' does not match the asset path ('return'). ANSWER: because our prediction is not a prediction but Daily Rebalanced P&L, where correctly predicted Negative move results in Positive P&L increment.
- We have done **worse** than buy and hold: simply investing $100 and holding the position.

FUDGED inverted sign applied to OLS prediction worked better at reproducing the asset price path. Whoa!

ANSWER: that's likely because of cancelled false negatives, ie, OLS produces a lot of negative predicitons which do not realise.

Remember OLS is an **invalid** regression model for binary dependent variable {0, 1}. **DO NOT** run OLS in your ML assignments and projects.

Logistic Classifier (in detail) // Support Vectors (comparison)

The one and only hyperparameter of interest is C, which relates to penalisation or soft/hard margins.

- **Logistic Classifier**

$C = 10^5$ implies nearly no L2 Penalty because parater set in **inverse** $\lambda = \frac{1}{C}$. Try strong penalisation C = 0.01 to 0.5 to 1 for L2 penalty.

The default penalisation is L2 which is known as Ridge Regression. L1 penalisation present is known as Lasso Regression. L1 vs L2 type penalty can be investigated but the answer is: impact of L1 penalisation is very strong, it zeroes-out out the coefficients fast.

- **SVM Classifier**

$C = 10^5$ for Support Vector Machines works out as Hard Margins -- the larger C parameter is the harder the margin. Prediction with Soft Margins is supposed to work better on financial time series.

```
1) Setting the margins hard and with the right choice of
SVM kernel function, all points withiin the training set
will be classified correctly. For our features of lagged
returns $C=10^3$ seems to achive that effect of 100%
accuracy -- see Two Outcomes of SVM section below.

2) Alpha coefficients are stored in an attribute,
dual_coef_. The support_ and support_vector_ are also
available for visualisation. The trailing underscore is a
scikit-learn convention that says that these attributes
are estimated from the data.
```

SVM classification requires much selectivity in features choosen because it becomes noticeably slow when you input more than a few columns (eg, 5-8 columns and computation is not instant). The separability of classes is problematic across multiple dimensions (features): you can see that by plotting features pairwise (2D) and applying different colors to different class labels. That SVM 2D plot with margins is different idea from the sigmoid plot presented below to analyse the predictive powers of Logistic Classifier (regressor). SVM with Hard Margin

---

Maximum Likelihood in Binomial Classification (Logistic Classifier/Regression)

Let's find an analytical solution to the maximum likelihood estimation problem for a mix of independent identically distributed Bernoulli draws in the regression (prediction) setting.

Each {0,1} outcome has a set of its own explanatory variables $\boldsymbol{X_i}$.

$$y_i|\boldsymbol{X_i} \sim \text{Bernoulli}(p_i)$$

$$\mathbb{E}\left[y_i|\boldsymbol{X_i}\right] = p_i$$

$$\Pr\left(y_i = 1, 0\right) = \begin{cases} p_i \\ 1 - p_i \end{cases}$$

$$f(y_i; p_i) = p_i^{y_i}(1-p_i)^{1-y_i}$$

Each outcome is determined by the probability of default p_i, which is unobserved (latent) in the regression model.

Logistic ML Part 1

The Bernoulli density above translates to the log-likelihood (contribution from one observation i).

$$\log L_i = \log f(y_i; p_i) = y_i \log p_i + (1 - y_i) \log(1 - p_i)$$

The joint log-likelihood for multiple events observed together and treated as independent, is given by the Product Rule of probabilties:

$$\begin{aligned} \log f(y_1, y_2, \ldots, y_N) &= \log \prod_{i=1}^{N_{obs}} f(y_i; p_i) \\ &= \sum_{i=1}^{N_{obs}} \log f(y_i; p_i) \end{aligned}$$

$$\log L = \sum_{i=1}^{N_{obs}} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right] \tag{1}$$

y_i is known from dataset.

p_i requires inverse of the link $p_i = g^{-1}(\boldsymbol{X_i}\boldsymbol{\beta'})$

Logistic ML Part 2

We can express Bernoulli density for a random variable $y = \{1, 0\}$ in a more canonical form -- as a member of the Exponential family of distributions.

$$f(y; p) = p^y (1-p)^{1-y} = \exp\left[y \log\left(\frac{p}{1-p} \right) + \log(1-p) \right]$$

Choice of a link function is \underline{the same for any categorical $\boldsymbol{Y}$}

$$g(p) = \log\left(\frac{p}{1-p} \right)$$

This is a logit function (different from logistic function!), which can be read as the ``log of odds''. The inverse of logit function is logistic function, which we are interested in.

$$p = \frac{1}{1 + e^{-g}}$$

Under the hood: the choice of Link Function

To adapt the linear regression to non-linear output variable $y_i = 0, 1$ or Binomial $y_i = 0, 1, 2, 3, 4, \ldots$ or in general case to probability we introduce a non-linear link funciton $y = g(p)$

$$\text{Dependent Varialbe } = \text{ Link Function} \left(\text{ Probability } \right)$$

$$\begin{aligned} \mathbf{Y} &= \boldsymbol{X}\boldsymbol{\beta}' + \boldsymbol{\epsilon} && (2) \\ PD \equiv \mathbb{E}[\mathbf{Y}|\mathbf{X}] &= \boldsymbol{X}\boldsymbol{\beta}' && (3) \\ g(p) &= \boldsymbol{X}\boldsymbol{\beta}' \qquad \text{and} && (4) \\ p &= g(\boldsymbol{X}\boldsymbol{\beta}')^{-1} && (5) \end{aligned}$$

$$\text{Probability } = \textbf{ Inverse Link Function} \left(\text{ Dependent Variable } \right)$$

- This covers default/no default $y_i = \{1, 0\}$ and ordinal ratings $y_i = 1, 2, 3, 4, 5$. In fact, response variable $\boldsymbol{Y}$ can have any distribution from Exponential family (quasi MLE).

- Linear part $\boldsymbol{\beta X}$ is linked to a non-linear, latent variable (probability).

$$p = g(\boldsymbol{X}\boldsymbol{\beta}')^{-1}$$

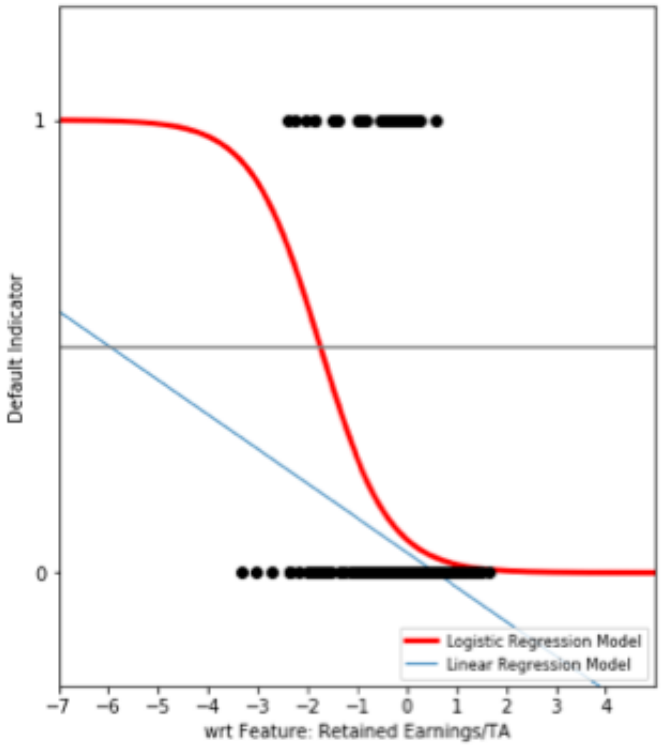
A link function is the clever bit that allows to convert a categorical event indicator $y_i = \{1, 0\}$ to the probability p_i

$$p_i = g^{-1}(\boldsymbol{X_i}\boldsymbol{\beta}')$$

$$\begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \Rightarrow \begin{pmatrix} g(\boldsymbol{X_1}\boldsymbol{\beta}')^{-1} \\ \vdots \\ g(\boldsymbol{X_n}\boldsymbol{\beta}')^{-1} \end{pmatrix} \Rightarrow \begin{pmatrix} \text{Prob}_1 \\ \vdots \\ \text{Prob}_n \end{pmatrix}$$

Let's make a step forward and say that the inverse of our link will be **the logistic sigmoid** function,

$$p = \frac{1}{1 + e^{-\boldsymbol{X}\boldsymbol{\beta}'}} = \frac{e^{\boldsymbol{X}\boldsymbol{\beta}'}}{1 + e^{\boldsymbol{X}\boldsymbol{\beta}'}}$$



In [119…
```
# Note the use dataset copy "data_adv copy"
# Remind ourselves data structure
cols
```

Out[119… `['ret_1', 'ret_2', 'ret_3', 'ret_4', 'ret_5']`

In [120…
```
data_adv[cols].head()
```

Out[120…

Date	ret_1	ret_2	ret_3	ret_4	ret_5
2010-10-08	-0.008148	-0.007225	0.029947	-0.006245	-0.000305
2010-10-11	0.011860	-0.008148	-0.007225	0.029947	-0.006245
2010-10-12	0.004631	0.011860	-0.008148	-0.007225	0.029947
2010-10-13	0.004720	0.004631	0.011860	-0.008148	-0.007225
2010-10-14	0.003553	0.004720	0.004631	0.011860	-0.008148

In [11]:
```
# Each supervised learning has Classifer and Regressor equivalent impleme

from sklearn import linear_model
from sklearn.svm import SVC
```

In [12]:
```
lm = linear_model.LogisticRegression(C = 1e5)

lm.fit(data_adv[cols], data_adv['return_sign'])

# FIT LOGISTIC
# Below output omits FULL SET of hyperparameters but for the Logistic Cla
# L1/L2 penalisation and its strength. Default choice is L2 penalty - Rid
```

Out[12]:
```
LogisticRegression
LogisticRegression(C=100000.0)
```

In [13]:
```
svcm = SVC(C = 1000, kernel='poly',  gamma='scale', probability=True) #ke

svcm.fit(data_adv[cols], data_adv['return_sign'])

# FIT SUPPORT VECTORS

# kernel='rbf', kernel='linear' lead to 100% in-sample prediction result.
# kernel='poly' is most time-consuming and gave us the desired variety

# gamma='scale' affects how kernel is executed, try gamma=0.1
# probability=True invokes something like 5-fold crossvalidation and make

# It is very difficult to trick SVM Classifier into bad (varied!) in-samp
#  -- desireable to generate such 'bad prediction' to get a sense in whic
#  -- instead possible to nudge SVM Classifier towards predicting all '1'

# The same numerical results out of SVM can't be guaranteed (for the same
# There is variability between sklearn versions.
```

Out[13]:
```
SVC(C=1000, kernel='poly', probability=True)
```

In [110…
```
def logistic_sigmoid(xb):
    return (1 / (1 + np.exp(-xb)))
```

In [64]:
```
#Procedure RELIES X_Features, Y_Response variables to be existing
def logistic_plot(X_min, X_max, FeatureName, FeatureBetaIdx):

    plt.clf() #clears the figure drawing space, nothing to do with classi
    fig, ax = plt.subplots(figsize=(18,10))  #fig = plt.figure(figsize=(1

    # 1. Plot two clusters of observations at Y={-1,1} on a scatter
    ax.scatter(data_adv[FeatureName], data_adv['logit_pred'], c=(data_adv

    # 2. Plot CALIBRATED sigmoid function -- with the correctly picked co
    X_Sim = np.linspace(X_min, X_max, 100) #fill in values for the range
    Y_Loss = logistic_sigmoid(X_Sim * lm.coef_[0,FeatureBetaIdx] + lm.int
    # Y_Loss = logistic_sigmoid(X_Sim * logit.coef_[0,FeatureBetaIdx] + l
    ax.plot(X_Sim, Y_Loss, color='red', linewidth=3) # plot sigmoid in Re

    plt.ylabel('Beta coeff value for sigmoid: -6.138', fontsize=22) # als
    plt.xlabel('wrt Feature: ' + FeatureName, fontsize=22)
    plt.xticks(np.linspace(X_min, X_max, num=10), fontsize=14)
    plt.yticks([-1, 1], fontsize=14)
    plt.ylim(-1.1, 1.1)
    plt.xlim(X_min, X_max) #Axe X range
    plt.legend(('Logistic Regression',),
           loc="lower right", fontsize=14)
    #plt.show()
    return ax
```

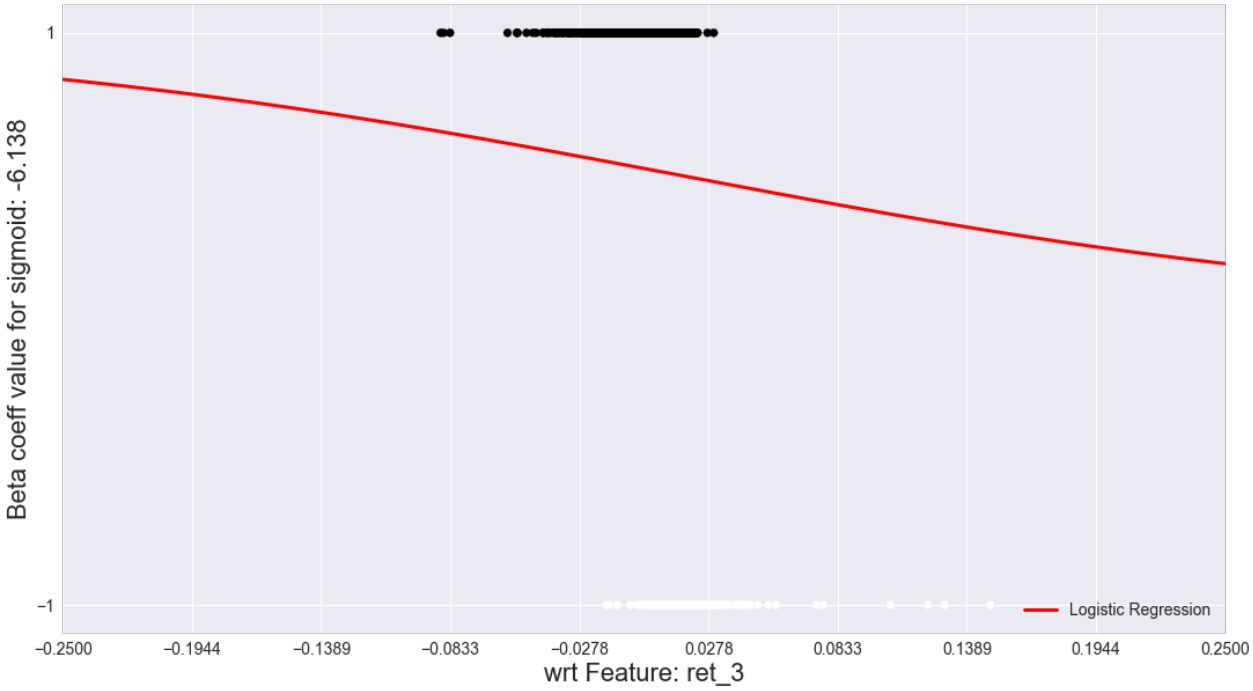
In [123…
```
# Error message below will remain due to difference in data types passed
# TypeError: 'float' object cannot be interpreted as an integer

logistic_plot(-0.25, 0.25, 'ret_3', 2)

# 'ret_1' has lm.coef_[0,0]

# 'ret_3' has lm.coef_[0,2], the coefficient for our GOOG is -6.13818103,
```

Out[123…
```
<matplotlib.axes._subplots.AxesSubplot at 0x7fe0dc157950>
<Figure size 432x288 with 0 Axes>
```



Above plot implements the inverse of our link will be **the logistic sigmoid** function,

$$p = \frac{1}{1 + e^{-\boldsymbol{x}\beta'}}$$

In [14]:
```
data_adv['logit_pred'] = lm.predict(data_adv[cols])
data_adv['logit_pred_backtest'] = data_adv['return'] * data_adv['logit_pr

data_adv['svm_pred'] = svcm.predict(data_adv[cols])
data_adv['svm_pred_backtest'] = data_adv['return'] * data_adv['svm_pred']
```

In [153…
```
data_adv.head(15)
```

Out[153…

Date	return	ret_1	ret_2	ret_3	ret_4	ret_5	return_
2010-10-08	0.011860	-0.008148	-0.007225	0.029947	-0.006245	-0.000305	
2010-10-11	0.004631	0.011860	-0.008148	-0.007225	0.029947	-0.006245	
2010-10-12	0.004720	0.004631	0.011860	-0.008148	-0.007225	0.029947	
2010-10-13	0.003553	0.004720	0.004631	0.011860	-0.008148	-0.007225	
2010-10-14	-0.004370	0.003553	0.004720	0.004631	0.011860	-0.008148	
2010-10-15	0.106028	-0.004370	0.003553	0.004720	0.004631	0.011860	
2010-10-18	0.026677	0.106028	-0.004370	0.003553	0.004720	0.004631	
2010-10-19	-0.016119	0.026677	0.106028	-0.004370	0.003553	0.004720	
2010-10-20	0.000231	-0.016119	0.026677	0.106028	-0.004370	0.003553	
2010-10-21	0.006582	0.000231	-0.016119	0.026677	0.106028	-0.004370	
2010-10-22	0.000885	0.006582	0.000231	-0.016119	0.026677	0.106028	
2010-10-25	0.006468	0.000885	0.006582	0.000231	-0.016119	0.026677	
2010-10-26	0.003381	0.006468	0.000885	0.006582	0.000231	-0.016119	
2010-10-27	-0.003446	0.003381	0.006468	0.000885	0.006582	0.000231	
2010-10-28	0.003446	-0.003446	0.003381	0.006468	0.000885	0.006582	

In [136…

```
# Logit
data_adv[['return','logit_pred_backtest']].cumsum().apply(np.exp).plot(fi
```



In [149…
```
# SVM (with poly kernel)
data_adv[['return','svm_pred_backtest']].cumsum().apply(np.exp).plot(figs
```



TWO OUTCOMES WITH SVC

- Support Vector classifier can cheat and predicted 1:1 and -1:-1 return sign match (positive and negative) **in sample**. This outcomes provides the upper possible boundary for the P&L where we excluded all false POSITIVE/true NEGATIVE and false NEGATIVE/true POSITIVE mistakes. On each day's bet we have gained only and grew the capital almost 9x-fold.

- The alternative and as well undesireable SVC prediction outcome is predicting just label 1. That achieves a repetition of the asset path return only since at all times,

  `data_adv['return'] * 1`

The outcomes can be said to be limiting or assymptotic.

In [146…
```
data_adv[['return','logit_pred_backtest','svm_pred_backtest']].cumsum().a
```



Takeaways and Further Steps

- We have discovered that supervised learning Classifiers delivered a reasonably good P&L generating predictions, over the multi-year period. That was unlike Linear Regression, prediction of which had to be **FUDGED** to its complete opposite in order to obtain a path, at least comparable to hold long only strategy.

- Betting strategy/rebalancing is assumed by **vectorised backtesting**. That at the start of the day you make a bet of +100% or -100% of the asset. Correctly predicted negative sign translates into Positive P&L.

  This is different from the long-only buy and hold startegy, for which you make one investment at time t_0 and no rebalancing.

  `cumsum(...).apply(np.exp)`

- Therefore, given the large enough count of daily bets (252 * 8 years) on direction predictions from Logistic/SVM/Decision Tree classifier, the resulting P&L is positive and growing. However, do not rush to a quick conclusion as to which is the better classifier: that would be dependent on historical regime (eg high volatility), frequency and the formulation of classification task (eg, for 15M data you might want to specify the task differently). Playing and tuning the hyperparameters will make impact, particularly with capricious methods

- ML tools use means that you will be building **a pipeline**. While not included in the code below due to complexity, the ultimate purpose of the pipeline is to implement bagging and boosting of your chosen classifier in order to make its prediction useful for trading and investment strategy.

    1. Feature scaling, importance ranking, and SOMs
    2. Hyperparameter tuning
    3. Bagging for regression, and use of ensemble methods (such as xgboost)

END OF DEMONSTRATION

AI

I have been looking for sensible AI tools to generate topical images for quant finance problems. In the past, I have used Craiyon free AI image generator tool with limited success. In 2023 Microsoft Copilot tool turned the page and generated images engage better. Two choosen examples below in response to my specific prompt "predicted daily return of S&P 500 index". Enjoy the art!





More Features from Price Information

- A typical set of fetures (return, momentum, moving average, exponential moving average) is given at the end. Experiment with different periods, however, be mindful of two aspects. 1) Autocorrelation begins to kick in for features computed over 5D, 10D periods (whether estimated with or without overlap on rolling basis, or crossvalidation style random sampling). 2) Technical analysts and financial media like to point out at EWA 50 and EWA 200 as long-run trend indicators. However, their relevance can be minimal for specific prediction of next day's move.

- Extend forecasting with the full scale of Technical Analysis best done/validated using the industry-scale libraries, like TA-Lib. Whenever possible thouhg, we would like to compute from the first principles -- in order to understand how TA indicators work, which class they belong to (eg, more momentum, or more vol), and what they are likely to signal.

    1. Exponential Moving Average (EMA)
    2. Bollinger Bands
    3. Volatility Estimators (yes, there is more than STD DEV and GARCH!)

Volatility: there is a variety of estimators, in addition to EWMA and GARCH. For prediction, one would like to study volatility estimation that focuses on capturing the realised variance (remember that historical sample standard deviation is like drawing random numbers from a hat!). The specific pointer here is *Drift-Independent Volatility Estimation Based on High, Low, Open, and Close Prices* by Yang and Zhang.

- The classic introduction to forecasting can be found in *Forecasting Methods and Applications* by Hyndman, Makridakis, and Wheelwright. The textbook diligently considers use of lagged returns, each relevant test such as F-statistic, ARIMA and how to implement de-trending and seasonality adjustments. Examples and focus on macro-economic, quarterly data (not very HF). But, it's known as Wiley's "Bible of Forecasting".

Great news there is the newer ONLINE VERSION Forecasting: Principles and Practice by Hyndman and Athanasopoulos. You can go right into the following sections,

relevant to quant work with time series.

1) Section 2.7 Lag plots and Section 2.8 Autocorrelation

2) Section 3.3 Moving Averages

3) Section 5.7 Residual Diagnostics (Facebook stock price), includes Box-Pierce and portmanteau tests

Section 5.7 link and many more.

In [ ]:
```
def createFeatures(df):
#    df = df.rename(columns={df.columns[0]: 'price'})
#    df['return'] = df.pct_change()
#    df['log_return'] = np.log(df['price']/df['price'].shift(periods=1))
#    df['sign'] = df['log_return'].apply(lambda x: 0 if x<0 else 1)

    df['return_1d']= df['return'].shift(periods=1)
    df['return_2d']= df['return'].shift(periods=2)
    df['return_5d']= df['return'].shift(periods=5)

    df['momentum_1d']=df['price']-df['price'].shift(periods=1)
    df['momentum_2d']=df['price']-df['price'].shift(periods=2)
    df['momentum_5d']=df['price']-df['price'].shift(periods=5)

    df['MA_5d']=df['price'].rolling(5).mean()
    df['MA_10d']=df['price'].rolling(10).mean()
    df['MA_20d']=df['price'].rolling(20).mean()
    df['MA_50d']=df['price'].rolling(50).mean()

    df['EMA_5d']=df['price'].ewm(5, adjust=False).mean()
    df['EMA_10d']=df['price'].ewm(10, adjust=False).mean()
    df['EMA_20d']=df['price'].ewm(20, adjust=False).mean()
    df['EMA_50d']=df['price'].ewm(50, adjust=False).mean()

    return df
```

Classifier Output

- Consider the accuracy and pattern of prediction *within each class*. This is necessary and can be done in the form of the common tools to check the output of ML Classifiers (Logistic/SVM/Decision Tree):

    1. **confusion matrix** and **area under the ROC curve**;

2. investigate the Recall for negative returns, ie, False Negatives problem.

- Consider advantages and disadvantages of moving onto multinomial classification $\{-1, 0, 1\}$. For example, what would you do if most of observations (daily return) will fall into category of 'no move'. We have provided some answers to 'no move'/small move problem at the start of the demo.

In [18]:
```
# Metrics
from sklearn.metrics import (precision_recall_curve,
                             roc_curve,
                             RocCurveDisplay,
                             ConfusionMatrixDisplay
                            )
from sklearn.metrics import (accuracy_score,
                             f1_score,
                             recall_score,
                             precision_score,
                             roc_auc_score,
                             auc
)
from sklearn.metrics import (classification_report,
                             confusion_matrix
)
```

In [ ]:
```
# Logistic Classifier fit method reminder

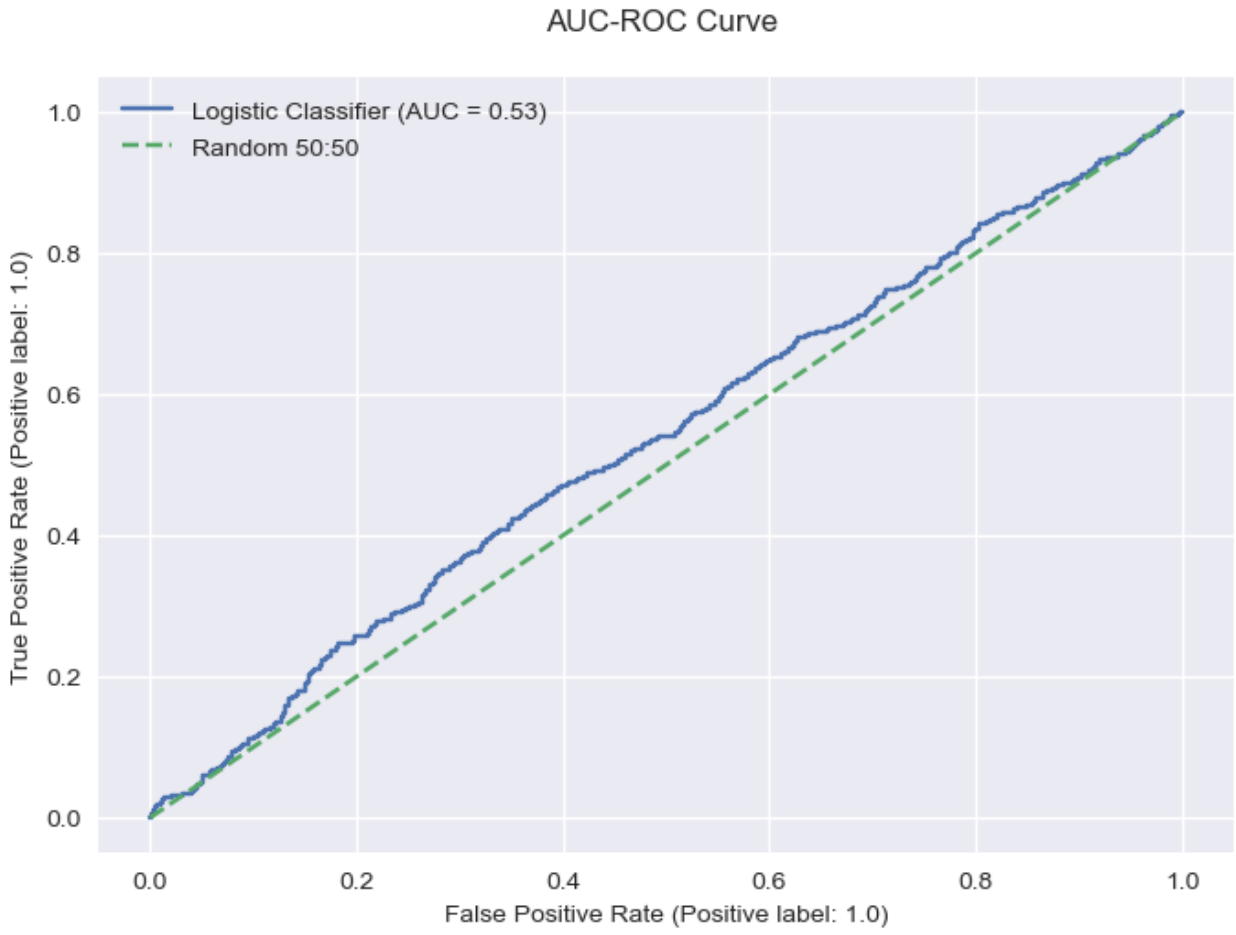
# lm.fit(data_adv[cols], data_adv['return_sign'])

X_train = data_adv[cols]

y_train = data_adv['return_sign']

# Extract Probabilities from Classifier -- computed with logistic functio
probs = lm.predict_proba(data_adv[cols])
```

In [27]:
```
# Display ROC Curve
disp = RocCurveDisplay.from_estimator(
            lm,
            X_train,
            y_train,
            name='Logistic Classifier')
plt.title("AUC-ROC Curve \n")
plt.plot([0,1],[0,1],linestyle="--", label='Random 50:50')
plt.legend()
plt.show()
```



In [28]:
```
# Display ROC Curve alternative method

preds1 = probs[:, 0]
preds2 = probs[:, 1]

fpr1, tpr1, threshold1 = roc_curve(y_train, preds1, pos_label=-1)
roc_auc1 = auc(fpr1, tpr1)

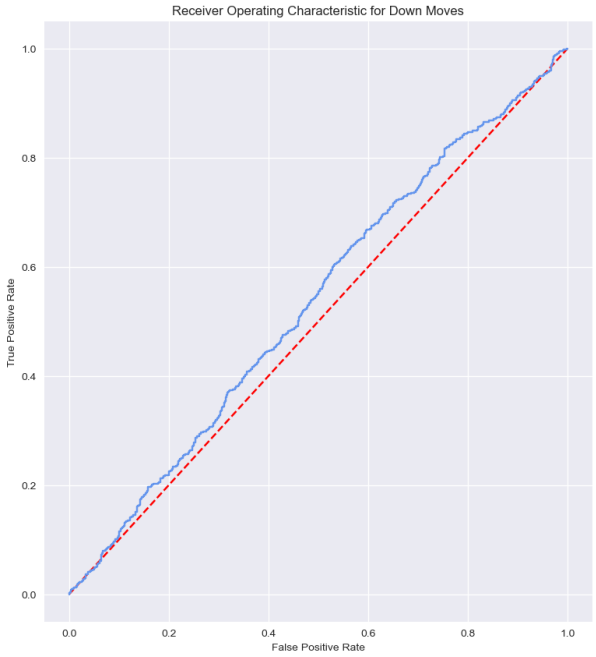
fpr2, tpr2, threshold2 = roc_curve(y_train, preds2, pos_label=1)
roc_auc2 = auc(fpr2, tpr2)
```

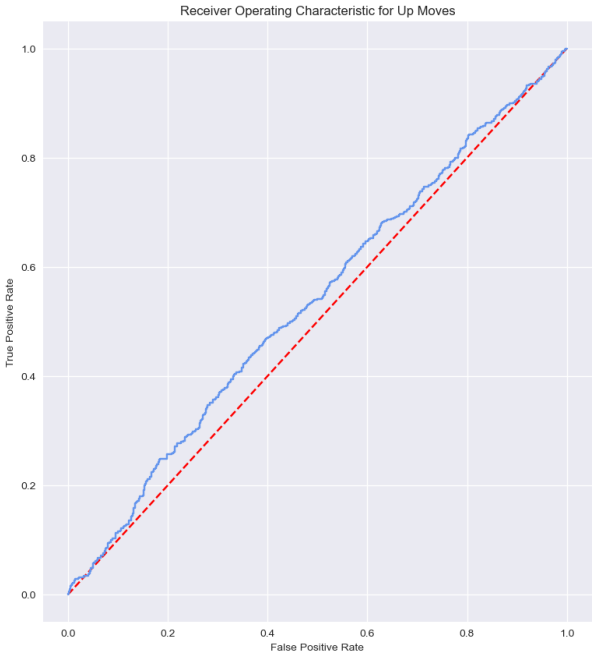
In [29]:
```
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(20,10))

ax[0].plot([0, 1], [0, 1], 'r--')
ax[0].plot(fpr1, tpr1, 'cornflowerblue', label=f'AUC = {roc_auc1:0.2}')
ax[0].set_title("Receiver Operating Characteristic for Down Moves")
ax[0].set_xlabel('False Positive Rate')
ax[0].set_ylabel('True Positive Rate')

ax[1].plot([0, 1], [0, 1], 'r--')
ax[1].plot(fpr2, tpr2, 'cornflowerblue', label=f'AUC = {roc_auc2:0.2}')
ax[1].set_title("Receiver Operating Characteristic for Up Moves")
ax[1].set_xlabel('False Positive Rate')
ax[1].set_ylabel('True Positive Rate')
```

Out[29]:
```
Text(0, 0.5, 'True Positive Rate')
```





END OF EXTRAS

In [ ]: